ScholarTrack: Technical Report

Module: XJCO3011 Web Services and Web Data
Author: Nuo Chen
Leeds ID: 201690992 | SWJTU ID: 2022116018
April 2026

GitHub repository	https://github.com/Nuo-cl/XJCO3011_CW1
API Documentation	https://github.com/Nuo-cl/XJCO3011_CW1/blob/main/docs/api_documentation.pdf
Technical Report	https://github.com/Nuo-cl/XJCO3011_CW1/blob/main/docs/ScholarTrack_Technical_Report.pdf
Presentation slides	https://github.com/Nuo-cl/XJCO3011_CW1/blob/main/docs/ScholarTrack_Presentation.pptx

1. Introduction

ScholarTrack is a RESTful Web API designed for researchers to discover **arXiv** papers, manage a personal research library, take structured notes, and receive personalized paper recommendations. The system also provides AI-powered semantic search across papers and notes using **Retrieval-Augmented Generation (RAG)** techniques, and exposes its functionality to AI assistants via the **Model Context Protocol (MCP)**.

The API comprises 25 endpoints across seven feature modules, supporting the full research workflow from paper discovery to knowledge organisation. The table below summarises all modules with their REST API endpoints and MCP tool counterparts.

Module	REST API Endpoints	MCP Tools
F1: Auth	POST register, POST login GET/PUT /users/me	register_user update_profile
F2: Discovery	GET /papers/search GET /papers/trending GET /papers/discover GET /papers/{id}	search_papers get_trending_papers discover_papers
F3: Library	POST/DELETE /papers/{id}/save GET /library, GET /tags POST/DELETE .../tags	save_paper list_library
F4: Notes	POST/GET /notes GET/PUT/DELETE /notes/{id} GET /papers/{id}/notes	create_note
F5: Recommend	GET /recommendations/daily	get_daily_recommendations
F6: Search	POST /search/papers POST /search/notes POST /search/all	search_notes search_knowledge

Table 1: Feature modules with 25 REST endpoints and 11 MCP tools

2. Design and Architecture

2.1 Overall Architecture

ScholarTrack follows a **layered architecture** with clear separation of concerns:

- **Routes Layer —** Flask Blueprints handling HTTP request/response, input validation, and HATEOAS link generation.
- **Service Layer —** Business logic encapsulated in ArxivService, ChromaDBService, RecommendationService, decoupled from HTTP concerns.
- **Data Layer —** SQLAlchemy ORM models with SQLite for persistent storage, ChromaDB for vector embeddings.

This separation allows the MCP server to reuse the service layer directly without duplicating business logic, and makes each layer independently testable.

2.2 Application Factory Pattern

The application uses Flask's **app factory pattern** (**create_app**), which provides several advantages (Pallets Projects, 2010):

- Multiple configurations (development, testing, production) can coexist without code changes.
- The testing suite creates isolated application instances with in-memory databases.
- Circular import issues are avoided by deferring extension initialisation.

2.3 RESTful API Design

The API adheres to REST principles (Fielding, 2000):

- **Resource-oriented URLs** — nouns represent resources (e.g., /api/papers, /api/notes), HTTP methods represent actions.
- **Stateless communication** — each request carries its own authentication token (JWT).
- **HATEOAS** — every response includes _links with related resource URIs, enabling clients to navigate the API without hardcoded paths.
- **Consistent error format** — all errors return {"error": "snake_case_type", "message": "..."} with appropriate HTTP status codes.

2.4 Data Model

The database schema consists of six tables:

Model	Purpose
User	Account credentials and preferences
Paper	Cached arXiv paper metadata
UserPaper	Many-to-many join for user collections
Tag	User-defined labels
UserPaperTag	Tag assignments to saved papers
Note	Research notes linked to papers

Key design decisions:

- **Papers are read-only from arXiv** — the system caches paper metadata locally on first access but does not allow user uploads.
- **Tags attach to UserPaper, not Paper** — different users can tag the same paper differently without interference.
- **Notes cascade-delete vectors** — deleting a note automatically removes its associated ChromaDB vectors.

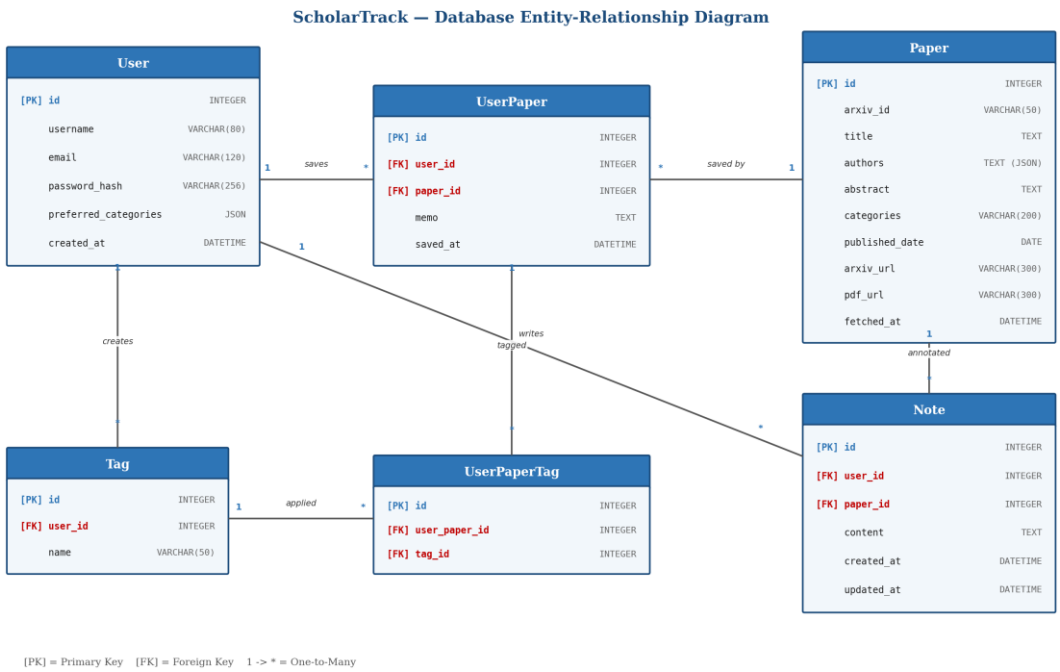

Figure 1: Database Entity-Relationship Diagram

3. Technology Stack Justification

- **Python —** The arXiv Python client (arxiv==2.2.0) and ChromaDB SDK are both Python-native, making integration straightforward. Node.js or Go alternatives would require custom HTTP wrappers (Python Software Foundation, 2024).
- **Flask —** Selected for its minimalism and explicit wiring (Pallets Projects, 2010). Django was rejected for its monolithic ORM/admin; FastAPI was considered but Flasgger provides equivalent OpenAPI generation.
- **SQLite + SQLAlchemy —** SQLite requires zero configuration, ideal for coursework. SQLAlchemy 2.0 provides database-agnostic migration support for future PostgreSQL scaling (Bayer, 2012).
- **ChromaDB —** Embeds all-MiniLM-L6-v2 by default, eliminating external model deployment (Chroma, 2024). PersistentClient for production, EphemeralClient for testing. FAISS rejected for lacking metadata filtering; Pinecone/Weaviate for cloud dependencies.
- **Authentication —** Stateless JWT via flask-jwt-extended with 24h expiry (Jones, Bradley and Sakimura, 2015). Flask-Limiter on auth endpoints (register: 5/min, login: 10/min). Passwords hashed with pbkdf2:sha256.

4. Core Feature Implementation

4.1 Data Access: Local Caching with On-Demand Fetching

arXiv enforces strict rate limits (~1 req/3s), making real-time queries impractical. The system caches papers in **SQLite** on first access (**arXiv ID** as dedup key, version suffixes stripped) and simultaneously indexes abstracts in **ChromaDB** for semantic search. **SQLite** was chosen over **Redis** because the single-server deployment needs no separate cache service, and cached data persists across restarts at zero operational cost. The trade-off is potential staleness — acceptable because arXiv papers are immutable once published.

Date constraints are embedded directly via arXiv's **submittedDate** syntax rather than filtering client-side — adopted after discovering that wildcard queries (**all:**) caused intermittent HTTP 500 errors. A **MAX_ARXIV_FETCH=100** cap and **delay_seconds=1.0** respect fair-use guidelines.

4.2 User Isolation and Multi-Tenancy

All user data is fully isolated by design. Tags attach to **UserPaper** (not Paper) via a **three-table pattern** (Tag → UserPaperTag → UserPaper), allowing different users to tag the same paper independently. A simpler JSON-array alternative was rejected because it prevents efficient tag-based filtering queries. Notes are isolated in **ChromaDB** via **user_id** metadata filtering, preventing cross-user leakage in semantic search. **JWT-only** authentication (no server-side sessions) simplifies deployment at the cost of inability to revoke tokens before expiry. The trade-off of full isolation is that collaborative features are unsupported — a deliberate scope decision to simplify access control.

4.3 Recommendation Algorithm: Hybrid Similarity-Freshness

Cold start (no saved papers): trending papers from preferred categories. **Warm start**: 70% **ChromaDB** semantic similarity against saved abstracts + 30% category-based freshness, with already-saved papers excluded. Pure similarity would create filter bubbles; pure freshness ignores interests — the **70/30 split** balances relevance and diversity. **Collaborative filtering** was considered but rejected due to data sparsity in a system where cross-user patterns are unavailable.

4.4 Dual-Collection Semantic Search

Two ChromaDB collections: **paper_abstracts** (shared, global) and **user_notes** (per-user, isolated). Papers are public resources; notes are private. Separate collections enforce access policies structurally — no authorization check needed on paper searches, while note searches always filter by **user_id**. Merging into one collection would risk accidental cross-user leakage. The **/api/search/all** endpoint queries both in parallel, normalises scores to **1−cosine_distance**, and merges by relevance with a **type** field for client differentiation.

4.5 API Design and REST Maturity

The API targets **Richardson Maturity Level 3**: resource URIs, HTTP verb semantics, and **HATEOAS** _links in every response for client-driven discovery. All errors use a consistent JSON format via a custom **APIError** exception, covering HTTP errors (400–500) and JWT errors to prevent raw HTML leaks. Rate limiting (**Flask-Limiter**) targets auth endpoints only (register: 5/min, login: 10/min) for credential-stuffing protection; general throttling is deferred to a reverse proxy in production.

4.6 Service Layer as Shared Backbone

Both REST routes and the **MCP server** (**FastMCP** SDK, 11 tools) call the same **service layer**. An alternative — MCP calling REST via HTTP — was rejected because it adds latency and requires a running server. Instead, MCP bootstraps a **Flask app context** and calls services directly. The trade-off is tighter coupling to Flask's lifecycle, acceptable for a monolithic deployment where both interfaces share the same process.

5. Testing

5.1 Approach

The test suite uses **pytest** with the following strategy:

- **Integration tests —** API endpoints tested via Flask's test client with an in-memory SQLite database and ephemeral ChromaDB instance.
- **Fixture-based setup —** conftest.py provides reusable fixtures (app, client, authenticated headers, sample data) to reduce test duplication.

5.2 Coverage

Component	Coverage
Models	95–100%
Routes	75–91%
Services	41–95%
Overall	80%

The **60 tests** cover all critical paths: authentication flows, CRUD operations, access control (**403** for cross-user access), input validation (**400** for missing fields), conflict handling (**409** for duplicates), and recommendation strategy correctness.

The lower coverage in arxiv_service.py (39%) reflects the difficulty of testing external API calls without network access; the service is tested indirectly through the paper endpoints using pre-cached data.

6. Challenges and Limitations

Key technical challenges encountered during development and their resolutions:

Challenge	Problem	Solution
ChromaDB file locking	PersistentClient holds file lock on Windows, causing PermissionError in test teardown.	Use EphemeralClient (in-memory) for testing via CHROMADB_PERSIST=False flag.
JWT identity mismatch	Login stored user ID as string; test fixtures used integers, causing 401 errors.	Test fixtures obtain tokens via the actual login endpoint instead of manual creation.
arXiv rate limiting	Strict rate limits (~1 req/3s) cause timeouts during development.	Cache papers in SQLite on first fetch; check local DB before API calls.
arXiv query instability	Wildcard queries (all:*) caused intermittent HTTP 500 from arXiv API.	Embed date constraints via submittedDate range syntax; remove wildcard terms.

Known limitations of the current implementation:

- **Single-user focus —** not stress-tested for concurrent writes; SQLite's single-writer may be a bottleneck.
- **Abstract-only indexing —** full-text PDF content not indexed; only abstracts are embedded in ChromaDB.
- **Stateless JWT —** tokens valid for 24h with no revocation; production would need a Redis-backed blacklist.
- **arXiv pagination —** iterator-based API lacks native offset; deeper pages waste network requests. Capped at 100.
- **Limited DoS protection —** rate limiting on auth endpoints only; production needs reverse proxy throttling.

Future work would focus on PostgreSQL migration for concurrent access, full-text PDF indexing via PyMuPDF, domain-specific embeddings (e.g., SPECTER2), and Docker-based deployment with CI/CD pipelines.

7. GenAI Declaration

This project was developed with the assistance of **Claude Code** (Anthropic, 2025), an AI-powered coding assistant. Claude Code was used for:

- **Architecture planning —** brainstorming feature decomposition and endpoint design.
- **Code implementation —** generating route handlers, service classes, and database models.
- **Test writing —** creating pytest test suites for each feature module.
- **Documentation —** generating API docstrings, README, and this technical report draft.
- **Debugging —** diagnosing issues such as the ChromaDB file lock problem and JWT identity type mismatch.

The development workflow used Claude Code's agent team feature, with a team lead coordinating developer and reviewer agents. All AI-generated code was reviewed, tested, and validated before inclusion. The architectural decisions and technology choices reflect the author's understanding and judgement, informed by AI suggestions. Detailed disclosure is provided in the appendix.

References

Anthropic. 2025. *Claude Code*. [Online]. [Accessed 3 April 2026]. Available from: https://claude.ai/code

Bayer, M. 2012. *SQLAlchemy*. [Online]. [Accessed 3 April 2026]. Available from: https://www.sqlalchemy.org/

Chroma. 2024. *ChromaDB: the AI-native open-source embedding database*. [Online]. [Accessed 3 April 2026]. Available from: https://www.trychroma.com/

Fielding, R.T. 2000. *Architectural styles and the design of network-based software architectures*. PhD thesis. University of California, Irvine.

Jones, M., Bradley, J. and Sakimura, N. 2015. JSON Web Token (JWT). *RFC 7519*. [Online]. Internet Engineering Task Force. [Accessed 3 April 2026]. Available from: https://datatracker.ietf.org/doc/html/rfc7519

Pallets Projects. 2010. *Flask: a Python microframework*. [Online]. [Accessed 3 April 2026]. Available from: https://flask.palletsprojects.com/

Python Software Foundation. 2024. *Python 3.11 documentation*. [Online]. [Accessed 3 April 2026]. Available from: https://docs.python.org/3.11/

Appendix A: Generative AI Usage Disclosure

A.1 Tools Used

Tool	Provider	Version / Model	Access Method
Claude Code	Anthropic	Claude Sonnet 4 (via API)	VS Code terminal agent
GitHub Copilot Chat	GitHub / Microsoft	Claude Opus 4.6	VS Code extension
GitHub Copilot	GitHub / Microsoft	Default model	VS Code inline completions

Claude Code was the primary tool (~90% of AI-assisted development), operating as an agentic coding assistant capable of reading/writing files and executing multi-step tasks autonomously. **GitHub Copilot Chat** was used for targeted questions and document generation.

A.2 Usage Methodology

AI was used **systematically** via a **Team Lead + Developer/Reviewer agent** workflow. The Team Lead decomposed tasks, delegated large independent blocks to Developer agents, and assigned Reviewer agents for milestone quality checks. Project context was maintained through **CLAUDE.md** (project guide) and specification documents loaded into every session, ensuring consistent, grounded AI outputs.

A.3 Usage by Development Phase

Phase	AI Usage	Human Judgement
Architecture & Planning	Feature decomposition into 7 modules. Data model design (6 tables + 2 ChromaDB collections). API endpoint structure (25 endpoints).	Technology selection rationale. Partition key and isolation strategy. Scope decisions (what NOT to build).
Implementation	Route handlers, service classes, models, MCP server tools, boilerplate code.	Design trade-offs (SQLite vs Redis, dual collections, 70/30 split). ArXiv API workarounds.
Testing	pytest suites for all 5 modules. Edge case identification.	Test strategy. Coverage targets. Manual API testing.
Debugging	ChromaDB file lock diagnosis. JWT identity type mismatch. arXiv query instability.	Root cause verification. Solution selection. Regression testing.
Documentation	README, Swagger docstrings, technical report, this disclosure.	Accuracy verification against code. Design rationale articulation.

A.4 Critical Analysis

Where GenAI excelled:

- **Pattern code** — boilerplate code (route handlers, models, fixtures) generated from specifications.
- **Cross-file consistency** — uniform HATEOAS links, error format, JWT decorators across all endpoints.
- **Error diagnosis** — ChromaDB file lock and JWT type mismatch diagnosed from error traces.

Where GenAI struggled:

- **Hallucinated features** — SM-2/flashcard references in the report that did not exist in code (14 corrections).
- **API-specific behaviour** — arXiv iterator pagination and wildcard query instability not anticipated by AI.
- **Scope creep** — tendency to suggest features beyond scope; required explicit constraints.

Decisions requiring human judgement:

- **Architecture** — SQLite over Redis (simplicity vs performance); three-table tags vs JSON arrays.
- **Algorithm design** — 70/30 similarity-freshness split; rejection of collaborative filtering.
- **Integration strategy** — MCP direct service calls vs HTTP forwarding; full user isolation vs collaboration.

A.5 Conversation Log Samples

More detailed conversation logs are available in the project repository (docs/conversation_log_full.html). Below are representative excerpts.

Sample — Testing a Feature Gap: From Exploration to Implementation

```
User: For testing the scenario of creating a brand-new user from scratch, what would the steps be on
      the API side vs the MCP side? Please describe the steps only — do not make any changes yet.
Agent: API side (HTTP):
       1. Register:
          POST /api/auth/register  {"username":"testuser","email":"...","password":"..."}
       2. Log in, get token:
          POST /api/auth/login  {"username":"testuser","password":"..."}
       3. Set preferred categories:
          PUT /api/users/me  (Bearer token)  {"preferred_categories":["cs.AI"]}
       4. Call any endpoint with the token as normal.
       MCP side:
       1. Register via API first — MCP has no registration tool.
          Note the returned user_id.
       2. Restart MCP server: python mcp_server.py --user-id <new_id>
       3. Set preferred categories — MCP currently has no update_profile tool.
          Options: call PUT /api/users/me via API, or add an MCP tool.
       4. Then test the full flow: get_daily_recommendations
          → cold-start hint → search/discover → save_paper
          → call recommendations again to observe strategy change.
       Core difference: the API side has built-in user registration and
       profile management; the MCP side is missing these two capabilities
       and currently depends on the API to create users and set preferences.
User: If we want MCP to be self-contained, we'll need registration and user profile editing. How
      exactly would that work?
Agent: The MCP server already has a Flask app context and direct DB access,
       so two tools suffice:
       register_user(username, email, password)
       - Create a User record inside _get_app().app_context()
         (password hashed with werkzeug generate_password_hash, pbkdf2:sha256)
       - Check username/email uniqueness; return an error on conflict
       - On success, auto-switch _user_id to the new user
       - Return {user_id, username, message}
       update_profile(preferred_categories, email)
       - Fetch User by current _user_id, update supplied fields
       - preferred_categories: comma-separated string -> list stored as JSON
       - Return updated profile
       Both tools sit in mcp_server.py alongside the existing tools.
       Thin wrappers — just direct reads/writes on the User model.
       No service layer or route changes needed.
       Shall I implement this?
User: Yes, please implement it.
Agent: [reads mcp_server.py]
       [adds 65 lines: register_user and update_profile with input
        validation, uniqueness checks, and password hashing]
       Verifying MCP server loads correctly:
       > python -c "import mcp_server; ..."
       ['register_user', 'update_profile', 'search_papers', ...]  ✓
       Running full test suite to ensure nothing is broken:
       > pytest tests/ -q
       60 passed in 14.22s  ✓
       Two new MCP tools added:
       - register_user: creates account, auto-switches active user context
       - update_profile: updates preferred_categories and/or email
       MCP can now run fully independently:
       register_user → update_profile → get_daily_recommendations
       → discover_papers → save_paper → recommendations (warm start)
```

A.6 Declaration

I confirm that all Generative AI usage in this project has been disclosed above. All AI-generated code was reviewed, tested, and validated before inclusion. The architectural decisions and design trade-offs represent my own understanding and judgement, informed by AI suggestions.